

Classicmodels API paths

This notebook calls the `/customers`, `/orders`, and `/orderdetails` routes from `app/main.py` over HTTP using the `requests` package.

Prerequisite: Run the API from the repository root, e.g. `uvicorn app.main:app --reload --port 8000` (or `python -m app.main`). The default base URL is `http://127.0.0.1:8000`; override with the environment variable `API_BASE_URL` if you use another host or port.

Note: Cells that `POST`, `PUT`, or `DELETE` modify the `classicmodels` database. Each section is rerun-safe — every test row is cleaned up at the end of its section, and the create cells use sentinel ids that won't collide with sample data (customer 9999, order 99999, orderdetail (10100, 'S10_1949')).

Setup

```
In [1]: import os

import requests

BASE_URL = os.environ.get("API_BASE_URL", "http://127.0.0.1:8000").rstrip("/")

try:
    _health = requests.get(f"{BASE_URL}/health", timeout=5)
    _health.raise_for_status()
except requests.RequestException as exc:
    raise RuntimeError(
        f"Cannot reach API at {BASE_URL!r}. From the repo root run e.g. "
        "`uvicorn app.main:app --reload --port 8000`, then rerun this cell "
        "(or set API_BASE_URL if the server uses another host/port).")
    from exc

print("API reachable at", BASE_URL)
```

API reachable at http://127.0.0.1:8000

Customers (`/customers`)

The customers table has 122 rows; primary key is `customerNumber` (int, not auto-increment).

GET `/customers` — list/search

Optional query parameters: `customerName`, `city`, `state`, `country`, `salesRepEmployeeNumber` (equality template).

```
In [2]: resp = requests.get(f"{BASE_URL}/customers", timeout=30)
assert resp.status_code == 200, resp.text
data = resp.json()
assert "items" in data
print("count:", len(data["items"]))
data["items"][:3]
```

count: 122

```
Out[2]: [{'customerNumber': 103,
  'customerName': 'Atelier graphique',
  'contactLastName': 'Schmitt',
  'contactFirstName': 'Carine ',
  'phone': '40.32.2555',
  'addressLine1': '54, rue Royale',
  'addressLine2': None,
  'city': 'Nantes',
  'state': None,
  'postalCode': '44000',
  'country': 'France',
  'salesRepEmployeeNumber': 1370,
  'creditLimit': 21000.0},
 {'customerNumber': 112,
  'customerName': 'Signal Gift Stores',
  'contactLastName': 'King',
  'contactFirstName': 'Jean',
  'phone': '7025551838',
  'addressLine1': '8489 Strong St.',
  'addressLine2': None,
  'city': 'Las Vegas',
  'state': 'NV',
  'postalCode': '83030',
  'country': 'USA',
  'salesRepEmployeeNumber': 1166,
  'creditLimit': 71800.0},
 {'customerNumber': 114,
  'customerName': 'Australian Collectors, Co.',
  'contactLastName': 'Ferguson',
  'contactFirstName': 'Peter',
  'phone': '03 9520 4555',
  'addressLine1': '636 St Kilda Road',
  'addressLine2': 'Level 3',
  'city': 'Melbourne',
  'state': 'Victoria',
  'postalCode': '3004',
  'country': 'Australia',
  'salesRepEmployeeNumber': 1611,
  'creditLimit': 117300.0}]
```

```
In [3]: resp = requests.get(f"{BASE_URL}/customers", params={"country": "USA"}, time
assert resp.status_code == 200
items = resp.json()["items"]
assert all(c["country"] == "USA" for c in items)
```

```
print(f"USA customers: {len(items)}")
items[:3]
```

USA customers: 36

```
Out[3]: [{ 'customerNumber': 112,
  'customerName': 'Signal Gift Stores',
  'contactLastName': 'King',
  'contactFirstName': 'Jean',
  'phone': '7025551838',
  'addressLine1': '8489 Strong St.',
  'addressLine2': None,
  'city': 'Las Vegas',
  'state': 'NV',
  'postalCode': '83030',
  'country': 'USA',
  'salesRepEmployeeNumber': 1166,
  'creditLimit': 71800.0},
{ 'customerNumber': 124,
  'customerName': 'Mini Gifts Distributors Ltd.',
  'contactLastName': 'Nelson',
  'contactFirstName': 'Susan',
  'phone': '4155551450',
  'addressLine1': '5677 Strong St.',
  'addressLine2': None,
  'city': 'San Rafael',
  'state': 'CA',
  'postalCode': '97562',
  'country': 'USA',
  'salesRepEmployeeNumber': 1165,
  'creditLimit': 210500.0},
{ 'customerNumber': 129,
  'customerName': 'Mini Wheels Co.',
  'contactLastName': 'Murphy',
  'contactFirstName': 'Julie',
  'phone': '6505555787',
  'addressLine1': '5557 North Pendale Street',
  'addressLine2': None,
  'city': 'San Francisco',
  'state': 'CA',
  'postalCode': '94217',
  'country': 'USA',
  'salesRepEmployeeNumber': 1165,
  'creditLimit': 64600.0}]
```

GET /customers/{customerNumber}

Returns **404** if the customer does not exist.

```
In [4]: resp = requests.get(f"{BASE_URL}/customers/103", timeout=30)
assert resp.status_code == 200
resp.json()
```

```
Out[4]: {'customerNumber': 103,
        'customerName': 'Atelier graphique',
        'contactLastName': 'Schmitt',
        'contactFirstName': 'Carine ',
        'phone': '40.32.2555',
        'addressLine1': '54, rue Royale',
        'addressLine2': None,
        'city': 'Nantes',
        'state': None,
        'postalCode': '44000',
        'country': 'France',
        'salesRepEmployeeNumber': 1370,
        'creditLimit': 21000.0}
```

```
In [5]: missing = requests.get(f"{BASE_URL}/customers/999999", timeout=30)
        assert missing.status_code == 404
        missing.json()
```

```
Out[5]: {'detail': 'No customer with customerNumber 999999'}
```

POST /customers

Creates a customer; **customerNumber** must be supplied (the table does not auto-increment). Response body is the new id (JSON string).

```
In [6]: TEST_CUSTOMER_NUMBER = 9999

# Defensive cleanup so the cell is rerun-safe
requests.delete(f"{BASE_URL}/customers/{TEST_CUSTOMER_NUMBER}", timeout=30)

payload = {
    "customerNumber": TEST_CUSTOMER_NUMBER,
    "customerName": "Notebook Test Customer",
    "contactLastName": "Doe",
    "contactFirstName": "Jane",
    "phone": "555-0000",
    "addressLine1": "1 Test St",
    "city": "Testville",
    "country": "USA",
    "creditLimit": 5000.00,
}
resp = requests.post(f"{BASE_URL}/customers", json=payload, timeout=30)
assert resp.status_code == 200, resp.text
new_id = resp.json()
assert new_id == str(TEST_CUSTOMER_NUMBER)
new_id
```

```
Out[6]: '9999'
```

PUT /customers/{customerNumber}

Replaces the row identified by the path; **404** if the customer does not exist.

```
In [7]: update_body = {**payload, "phone": "555-9999", "city": "NewCity"}
resp = requests.put(f"{BASE_URL}/customers/{TEST_CUSTOMER_NUMBER}", json=update_body)
assert resp.status_code == 200, resp.text
assert resp.json() == {"updated": 1}
requests.get(f"{BASE_URL}/customers/{TEST_CUSTOMER_NUMBER}", timeout=30).json()
```

```
Out[7]: {'customerNumber': 9999,
'customerName': 'Notebook Test Customer',
'contactLastName': 'Doe',
'contactFirstName': 'Jane',
'phone': '555-9999',
'addressLine1': '1 Test St',
'addressLine2': None,
'city': 'NewCity',
'state': None,
'postalCode': None,
'country': 'USA',
'salesRepEmployeeNumber': None,
'creditLimit': 5000.0}
```

DELETE /customers/{customerNumber}

Returns **{"deleted": 0}** or **{"deleted": 1}**.

```
In [8]: resp = requests.delete(f"{BASE_URL}/customers/{TEST_CUSTOMER_NUMBER}", timeout=30)
assert resp.status_code == 200
assert resp.json()["deleted"] == 1

gone = requests.get(f"{BASE_URL}/customers/{TEST_CUSTOMER_NUMBER}", timeout=30)
assert gone.status_code == 404
gone.json()
```

```
Out[8]: {'detail': 'No customer with customerNumber 9999'}
```

Orders (/orders)

The orders table has 326 rows; primary key is **orderNumber** (int). Date fields (**orderDate**, **requiredDate**, **shippedDate**) round-trip as ISO-8601 strings (**YYYY-MM-DD**).

GET /orders — list / search

Optional query parameters: **status**, **customerNumber**, **orderDate**.

```
In [9]: resp = requests.get(f"{BASE_URL}/orders", timeout=30)
        assert resp.status_code == 200
        data = resp.json()
        print("count:", len(data["items"]))
        data["items"][:3]
```

count: 326

```
Out[9]: [{'orderNumber': 10100,
          'orderDate': '2003-01-06',
          'requiredDate': '2003-01-13',
          'shippedDate': '2003-01-10',
          'status': 'Shipped',
          'comments': None,
          'customerNumber': 363},
         {'orderNumber': 10101,
          'orderDate': '2003-01-09',
          'requiredDate': '2003-01-18',
          'shippedDate': '2003-01-11',
          'status': 'Shipped',
          'comments': 'Check on availability.',
          'customerNumber': 128},
         {'orderNumber': 10102,
          'orderDate': '2003-01-10',
          'requiredDate': '2003-01-18',
          'shippedDate': '2003-01-14',
          'status': 'Shipped',
          'comments': None,
          'customerNumber': 181}]
```

```
In [10]: resp = requests.get(f"{BASE_URL}/orders", params={"status": "Shipped"}, time
        assert resp.status_code == 200
        items = resp.json()["items"]
        assert all(o["status"] == "Shipped" for o in items)
        print(f"shipped orders: {len(items)}")
```

shipped orders: 303

GET /orders/{orderNumber}

```
In [11]: resp = requests.get(f"{BASE_URL}/orders/10100", timeout=30)
        assert resp.status_code == 200
        resp.json()
```

```
Out[11]: {'orderNumber': 10100,
          'orderDate': '2003-01-06',
          'requiredDate': '2003-01-13',
          'shippedDate': '2003-01-10',
          'status': 'Shipped',
          'comments': None,
          'customerNumber': 363}
```

```
In [12]: missing = requests.get(f"{BASE_URL}/orders/999999", timeout=30)
        assert missing.status_code == 404
        missing.json()
```

Out[12]: {'detail': 'No order with orderNumber 999999'}

POST /orders

Creates an order; **orderNumber** must be supplied. **customerNumber** must reference an existing customer (FK).

```
In [13]: TEST_ORDER_NUMBER = 99999

requests.delete(f"{BASE_URL}/orders/{TEST_ORDER_NUMBER}", timeout=30)

order_payload = {
    "orderNumber": TEST_ORDER_NUMBER,
    "orderDate": "2026-01-01",
    "requiredDate": "2026-01-15",
    "shippedDate": None,
    "status": "In Process",
    "comments": None,
    "customerNumber": 103,
}
resp = requests.post(f"{BASE_URL}/orders", json=order_payload, timeout=30)
assert resp.status_code == 200, resp.text
new_order_id = resp.json()
assert new_order_id == str(TEST_ORDER_NUMBER)
new_order_id
```

Out[13]: '99999'

PUT /orders/{orderNumber}

Update **status** + fill in **shippedDate** + add a comment.

```
In [14]: update_body = {**order_payload, "status": "Shipped", "shippedDate": "2026-01-10"}
resp = requests.put(f"{BASE_URL}/orders/{TEST_ORDER_NUMBER}", json=update_body, timeout=30)
assert resp.status_code == 200
assert resp.json() == {"updated": 1}
requests.get(f"{BASE_URL}/orders/{TEST_ORDER_NUMBER}", timeout=30).json()
```

```
Out[14]: {'orderNumber': 99999,
          'orderDate': '2026-01-01',
          'requiredDate': '2026-01-15',
          'shippedDate': '2026-01-10',
          'status': 'Shipped',
          'comments': 'delivered',
          'customerNumber': 103}
```

DELETE /orders/{orderNumber}

```
In [15]: resp = requests.delete(f"{BASE_URL}/orders/{TEST_ORDER_NUMBER}", timeout=30)
assert resp.status_code == 200
```

```

assert resp.json()["deleted"] == 1

gone = requests.get(f"{BASE_URL}/orders/{TEST_ORDER_NUMBER}", timeout=30)
assert gone.status_code == 404
gone.json()

```

Out[15]: {'detail': 'No order with orderNumber 99999'}

Order details (/orderdetails)

The orderdetails table has 2,996 rows; primary key is the **composite** (orderNumber, productCode). Single-resource paths use both columns:

/orders/{orderNumber}/orderdetails/{productCode}.

GET /orderdetails — list/search

Optional query parameters: orderNumber, productCode (each part of the composite key, used independently).

```

In [16]: resp = requests.get(f"{BASE_URL}/orderdetails", timeout=30)
assert resp.status_code == 200
data = resp.json()
print("count:", len(data["items"]))
data["items"][:3]

```

count: 2996

```

Out[16]: [{'orderNumber': 10100,
  'productCode': 'S18_1749',
  'quantityOrdered': 30,
  'priceEach': 136.0,
  'orderLineNumber': 3},
 {'orderNumber': 10100,
  'productCode': 'S18_2248',
  'quantityOrdered': 50,
  'priceEach': 55.09,
  'orderLineNumber': 2},
 {'orderNumber': 10100,
  'productCode': 'S18_4409',
  'quantityOrdered': 22,
  'priceEach': 75.46,
  'orderLineNumber': 4}]

```

```

In [17]: resp = requests.get(f"{BASE_URL}/orderdetails", params={"orderNumber": 10100})
assert resp.status_code == 200
items = resp.json()["items"]
assert all(d["orderNumber"] == 10100 for d in items)
print(f"line items on order 10100: {len(items)}")
items

```

line items on order 10100: 4


```
Out[17]: [{ 'orderNumber': 10100,
            'productCode': 'S18_1749',
            'quantityOrdered': 30,
            'priceEach': 136.0,
            'orderLineNumber': 3},
          { 'orderNumber': 10100,
            'productCode': 'S18_2248',
            'quantityOrdered': 50,
            'priceEach': 55.09,
            'orderLineNumber': 2},
          { 'orderNumber': 10100,
            'productCode': 'S18_4409',
            'quantityOrdered': 22,
            'priceEach': 75.46,
            'orderLineNumber': 4},
          { 'orderNumber': 10100,
            'productCode': 'S24_3969',
            'quantityOrdered': 49,
            'priceEach': 35.29,
            'orderLineNumber': 1}]
```

GET

`/orders/{orderNumber}/orderdetails/{productCode}`

Identity is the composite PK — both URL segments are required. **404** if the line item does not exist.

```
In [18]: resp = requests.get(f"{BASE_URL}/orders/10100/orderdetails/S18_1749", timeout=5)
         assert resp.status_code == 200
         resp.json()
```

```
Out[18]: { 'orderNumber': 10100,
            'productCode': 'S18_1749',
            'quantityOrdered': 30,
            'priceEach': 136.0,
            'orderLineNumber': 3}
```

```
In [19]: missing = requests.get(f"{BASE_URL}/orders/10100/orderdetails/NOPE", timeout=5)
         assert missing.status_code == 404
         missing.json()
```

```
Out[19]: { 'detail': "No order detail with {'orderNumber': 10100, 'productCode': 'NOPE'}"}
```

POST /orderdetails

Creates a line item. Both `orderNumber` and `productCode` must reference existing rows (FK). Response body is the JSON-encoded composite PK.

```
In [20]: TEST_OD_ORDER = 10100
         TEST_OD_PRODUCT = "S10_1949" # exists in products, not currently a line item
```

```
requests.delete(f"{BASE_URL}/orders/{TEST_OD_ORDER}/orderdetails/{TEST_OD_PRODUCT_CODE}")

od_payload = {
    "orderNumber": TEST_OD_ORDER,
    "productCode": TEST_OD_PRODUCT,
    "quantityOrdered": 5,
    "priceEach": 100.00,
    "orderLineNumber": 5,
}

resp = requests.post(f"{BASE_URL}/orderdetails", json=od_payload, timeout=30)
assert resp.status_code == 200, resp.text
new_od_id = resp.json()
print("new id:", new_od_id)
new_od_id
```

```
new id: {"orderNumber": 10100, "productCode": "S10_1949"}
```

```
Out[20]: '{"orderNumber": 10100, "productCode": "S10_1949"}'
```

PUT

/orders/{orderNumber}/orderdetails/{productCode}

URL params are the source of truth for identity — `orderNumber` / `productCode` in the body are silently ignored.

```
In [21]: update_body = {**od_payload, "quantityOrdered": 99, "priceEach": 50.00}
resp = requests.put(
    f"{BASE_URL}/orders/{TEST_OD_ORDER}/orderdetails/{TEST_OD_PRODUCT_CODE}",
    json=update_body,
    timeout=30,
)
assert resp.status_code == 200
assert resp.json() == {"updated": 1}
requests.get(f"{BASE_URL}/orders/{TEST_OD_ORDER}/orderdetails/{TEST_OD_PRODUCT_CODE}")
```

```
Out[21]: {'orderNumber': 10100,
'productCode': 'S10_1949',
'quantityOrdered': 99,
'priceEach': 50.0,
'orderLineNumber': 5}
```

DELETE

/orders/{orderNumber}/orderdetails/{productCode}

```
In [22]: resp = requests.delete(
    f"{BASE_URL}/orders/{TEST_OD_ORDER}/orderdetails/{TEST_OD_PRODUCT_CODE}",
    timeout=30,
)
assert resp.status_code == 200
assert resp.json()["deleted"] == 1

gone = requests.get(f"{BASE_URL}/orders/{TEST_OD_ORDER}/orderdetails/{TEST_OD_PRODUCT_CODE}")
```

```
assert gone.status_code == 404  
gone.json()
```

```
Out[22]: {'detail': 'No order detail with {'orderNumber': 10100, 'productCode': 'S10_1949'}}}
```